

UNIT-II

Classic RL Assignment: The Tic-Tac-Toe Challenge

Key Terms Used in This Unit

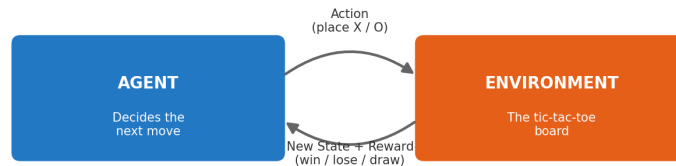
- **State:** a snapshot of the situation the agent is in — for tic-tac-toe, this is simply the current board layout.
- **Action:** a choice the agent can make from a state — here, which empty cell to place a mark in.
- **Reward:** a number the environment gives back after an action, showing how good or bad it was (e.g. +1 for a win, -1 for a loss, 0 for a draw or a normal move).
- **Policy:** the agent's strategy — the rule it uses to pick an action for each state. In Q-Learning, the policy is simply “pick the action with the highest Q-value.”
- **Episode:** one complete play-through, from the empty board to the game ending in a win, loss, or draw.

Tic-tac-toe is a useful teaching example because it is a small, finite, fully observable game: there are only a limited number of board states, the agent can always see the whole board, and there's no hidden randomness. This makes it possible to build a complete Q-table and see Q-Learning work from start to finish, which is much harder in larger real-world problems.

1. Recap: Dynamic Programming & Monte Carlo

Reinforcement Learning (RL) is about an agent learning to make good decisions by interacting with an environment, rather than being told the right answer directly. It tries an action, sees what happens, and gets a reward that tells it how good or bad that action was. Over many tries, it learns which actions lead to the best long-term outcome — in our case, winning at tic-tac-toe.

Recap: How a Reinforcement Learning Agent Learns



The agent tries a move, the board reacts, and a reward tells the agent if that move was good or bad. This repeats until the game ends.

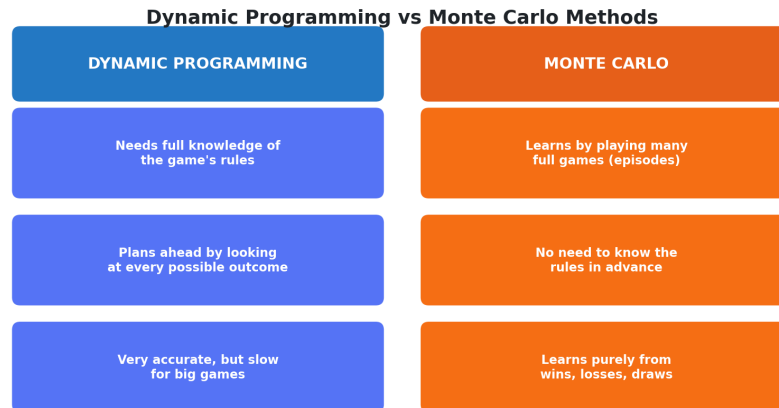
Figure 1: The basic agent – environment loop

Before Q-Learning, two classic (“tabular”) methods were used to solve this kind of problem:

- **Dynamic Programming (DP)** assumes the agent already knows the full “model” of the environment — all the rules, all possible states, and all outcomes of each move. It uses this knowledge to plan the best action mathematically, without needing to actually play any games.

- Monte Carlo (MC) methods don't need the rules in advance. Instead, the agent plays complete games (episodes) and learns purely from the final outcome — win, lose, or draw — updating its knowledge only after each game ends.

Q-Learning, covered next, borrows ideas from both: like Monte Carlo it learns from experience, but like Dynamic Programming it can update its knowledge after every single move instead of waiting for the whole game to finish.



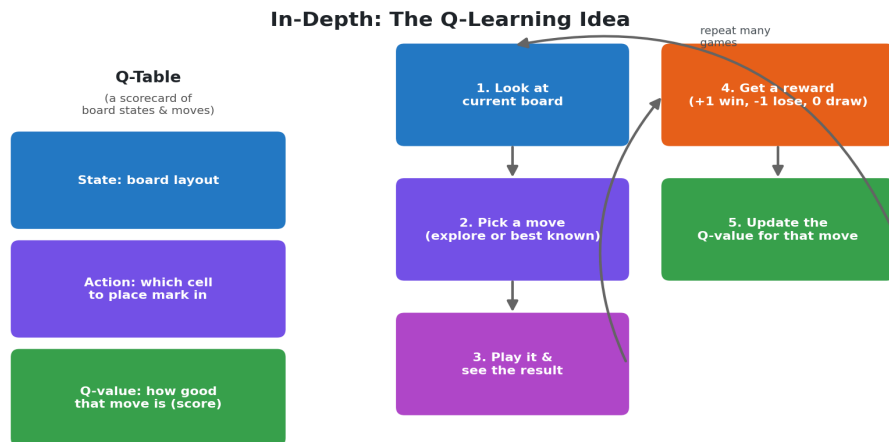
Both are classic ("tabular") RL methods — the building blocks that Q-Learning improves on.

Figure 2: Dynamic Programming vs Monte Carlo, side by side

2. In-Depth: The Q-Learning Algorithm

Q-Learning is a model-free RL method — the agent doesn't need to know the rules of the game beforehand. It learns entirely through trial and error, which is what makes it so useful for problems like tic-tac-toe where we want the agent to discover a good strategy on its own.

The core idea is the Q-value: a number that estimates how good it is to take a particular action from a particular state. All these numbers together form the Q-table — essentially a big scorecard with one row per board layout and one column per possible move.



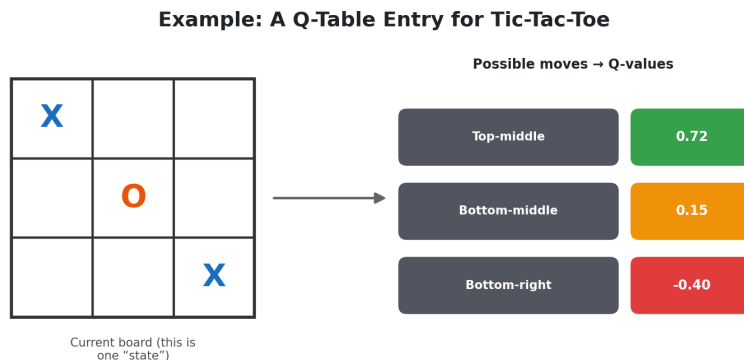
Q-Learning slowly fills in the Q-table with better and better scores through trial and error — no need to know the rules ahead of time.

Figure 3: The Q-Learning idea — Q-table + step-by-step loop

After every move, the agent updates the Q-value for that move using the Q-Learning update rule. In simple terms, the new score is a mix of the old score and the new information just received:

- New Q-value = Old Q-value + Learning Rate × (Reward + best future Q-value – Old Q-value)
- Learning Rate controls how much the new experience changes the old score (a small nudge vs. a big update).
- Discount Factor (hidden inside “best future Q-value”) controls how much the agent cares about future rewards vs. immediate ones.

Because the update looks at the best possible future Q-value, the agent effectively “looks ahead” one step, even though it never memorised the game's rules — it only ever learned from playing. This update rule is a simplified version of the Bellman equation, a core idea in RL that says the value of a state-action pair should equal the immediate reward plus the (discounted) value of the best action from the next state. Q-Learning is called “off-policy” because, when updating, it always assumes the best possible future move will be taken — even if the agent's actual next move (while exploring) turns out to be a different, more random one.



The agent checks the Q-values for this board and picks the move with the highest score — here, “Top-middle.”

Figure 4: A real Q-table example for one tic-tac-toe board

3. Training the Agent

Training means letting the agent play the game thousands of times (each full game is called an episode), so it can fill in its Q-table with accurate move scores. Early on, the Q-table is empty or random, so the agent's moves look clumsy; over many episodes the scores settle into sensible values and play improves.

Training the Q-Learning Agent to Play Tic-Tac-Toe



This loop repeats for thousands of games. Over time, the Q-table fills with better move scores and the agent improves.

Figure 5: The training loop for the tic-tac-toe agent

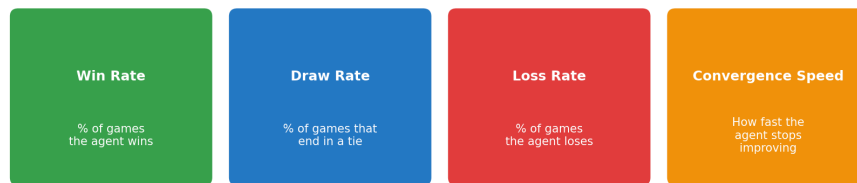
A key challenge during training is the explore vs. exploit trade-off — deciding whether to try something new or stick with what already seems to work:

- Explore: try a random, possibly untested move to discover new strategies the agent hasn't scored yet.
- Exploit: play the move with the best known Q-value, based on what's been learned so far.
- A common approach called epsilon-greedy explores with small probability (e.g. 10%) and exploits the rest of the time, so the agent keeps discovering while still playing sensibly.
- Good agents balance both — lots of exploring early, more exploiting later, so early mistakes don't get “locked in” before better moves are found.

4. Evaluation Metrics: Is the Agent Good?

Once trained, we can't just assume the agent is good — we need to test it against a fixed opponent (e.g. a random player or a perfect player) over many games and measure the results using simple metrics.

Evaluation Metrics: How Do We Know the Agent Is Good?



A well-trained agent should reach a high win/draw rate against a fixed opponent and rarely lose (since a perfectly played tic-tac-toe always ends in a draw).

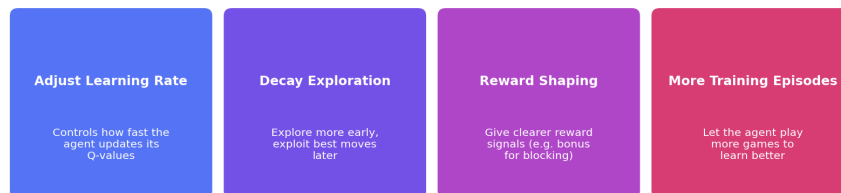
Figure 6: Four key metrics used to judge the agent

These metrics matter because tic-tac-toe has a known “perfect” outcome: if both players play optimally, every game ends in a draw. So a strong agent should almost never lose, should draw often against a perfect opponent, and should win consistently against a weaker (e.g. random) opponent. Convergence speed tells us how many training episodes were needed before performance stopped improving — useful for judging how efficient the training process was.

5. Fine-Tuning Strategies

If the agent isn't performing well after training, we can adjust its settings (often called hyperparameters) rather than redesigning the whole algorithm. Fine-tuning is usually an experimental process — change one setting, retrain, and compare the evaluation metrics.

Fine-Tuning Strategies: Making the Agent Play Better



Fine-tuning is about experimenting with these settings (“hyperparameters”) to get a smarter, more consistent agent.

Figure 7: Common ways to fine-tune the agent

- Learning rate: too high and the agent's scores swing wildly and never settle; too low and learning takes very long.
- Exploration decay: starting with high exploration and gradually lowering it helps the agent discover good strategies before locking into a routine.
- Reward shaping: giving extra reward signals (e.g. a small bonus for blocking an opponent's win) can speed up learning of good habits.

- More training episodes: simply letting the agent play more games often improves stability, especially for less common board states.

6. The Classic RL Assignment

The assignment brings everything together into one end-to-end project: build the tic-tac-toe game environment, build a Q-Learning agent, train it, evaluate it, and fine-tune it until it plays well.

Classic RL Assignment: Overall Pipeline

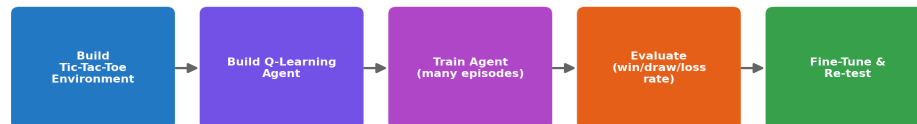


Figure 8: End-to-end pipeline for the assignment

- Step 1: Code the tic-tac-toe board and game rules — this defines the “environment” the agent will act in, including how states, actions, and rewards are represented.
- Step 2: Build the Q-Learning agent — set up the Q-table (or a data structure to hold it) and implement the update rule.
- Step 3: Train it by playing many games, against itself (self-play) or against a random player, while gradually shifting from exploring to exploiting.
- Step 4: Evaluate using win / draw / loss rate against a fixed test opponent to measure real performance.
- Step 5: Fine-tune settings such as learning rate and exploration decay, then re-test until performance is strong and stable.